Two Transformer Language Models Built From Scratch

Hindi (Higher-Resource) and Sindhi (Lower-Resource)

Neill Rawani | 2025202027

LMA Individual Project — Final Report

This report consolidates the whole project: headline results from Phases 1 and 2, and the reasoning finetuning and post-finetuning attention analysis of Phase 3 in full. Two decoder-only Transformers were built from scratch in PyTorch — **Model H** for Hindi (higher-resource) and **Model L** for Sindhi (lower-resource). They share no data, tokenizer, vocabulary or weights, and both had the same parameter budget (∼25M) and token budget (500M), so any difference between them comes from the languages and their corpora, not from capacity or compute.

1 Headline results

	Model H (Hindi)	**Model L** (Sindhi)
Training corpus available	504.8M tok	618.4M tok
Parameters / context length	24.22M / 512	24.48M / 1024
Bits per byte (test, ctx 512)	**0.5375**	**0.9584**
Train − validation loss gap	−0.12	+0.42
Reasoning, pretrained (0/4-shot)	0.0%	0.0%
Chance floor (random name)	37.2%	36.8%
Finetuned, in-distribution	**78.8%**	**73.7%**
Held-out names (generated / forced)	17.0% / 72.2%	14.8% / 70.7%
Bits per byte after finetuning	1.0111	1.9563

Table 1: Results across all three phases, with bits per byte at a matched 512-token context.

Model L needs 78.3% more bits per byte than Model H, despite having *more* training text available: the gap is about how varied and clean the data is, not how much there was. Both models learned the reasoning task from a standing start, having scored exactly zero beforehand; Model H learned it more completely, with almost all of the difference in multi-step chain problems rather than simple comparison, and both paid for it by roughly doubling their bits per byte on ordinary text.

Two results were unexpected. On Sindhi, perplexity claimed the test split was $2.6\times$ *easier* than validation, which cannot be true of two halves of one collection; chasing it uncovered encoding-corrupted text and moved the headline metric to bits per byte (§3.1). And on unseen names accuracy fell *below* chance — which turned out not to mean confusion at all (§4.2).

2 Phase 1 --- Corpus and Tokenizer

Hindi: 531,471,043 tokens over 2,214,401 documents. Bulk text from AI4Bharat Sangraha and IndicCorp V2; manual text from a custom streaming BZ2 parser over the Hindi Wikipedia XML dump plus scrapers for Gadyakosh, Dainik Bhaskar and Vikaspedia. Manual share $\geq$**29.4%, est.** ∼**43%**.

Sindhi: 651,508,880 tokens over 5,102,663 documents. Bulk text from HuggingFace collections (Cultuura, CC100, FineWeb2); manual text from the Sindhi Wikipedia dump and scrapers for the Sindhiana encyclopedia, Awami Awaz and the Sindh Salamat book library. Manual share $\geq$**9.1%, est.** $\sim$**38%** — high-quality bulk Sindhi data barely exists, so manual parsing was not optional.

Counting tokens. Each corpus was encoded with its own 16K Unigram tokenizer, counting every content line plus one `<|endoftext|>` per document boundary — exactly what the model is fed in Phase 2. Both were split 95/2.5/2.5 with a fixed seed.

Cleaning used six heuristic stages for both languages, with no ML models: a *length* filter (50–500,000 characters); a *symbol-ratio* filter (reject above 30% symbols and bracket punctuation, not counting Indic combining marks or the danda); a *repetition* filter (reject above 30% repeated word-level 5-grams); a *script* filter (at least 90% of alphabetic characters in the expected Unicode block); *exact deduplication* by SHA-256, applied globally *before* splitting so no document appears verbatim in two splits; and *fuzzy deduplication* by MinHash LSH. **79.6% of Hindi documents survived against 66.8% of Sindhi**, the difference mostly deduplication — 7.1% of Sindhi documents were exact duplicates against 2.8% for Hindi, which is what happens when scarce text is recirculated through several aggregated corpora.

	Hindi		**Sindhi**	
	Unigram	BPE	Unigram	BPE
Fertility (tok/word)	**1.3056**	1.3203	**1.2987**	1.3040
Token entropy (bits)	**10.535**	10.659	**10.770**	10.911
Vocab. utilisation	**97.39%**	95.52%	**98.23%**	97.09%
Unknown-token rate	0.0	0.0	0.0	0.0

Table 2: Tokenizer comparison on held-out text. **SentencePiece Unigram at 16,000 slots** was chosen for both, with byte fallback enabled.

Why Unigram. The fertility margin alone is thin; the real evidence is *token entropy*, since a less surprising token stream directly lower-bounds the perplexity the language model can reach, and *vocabulary utilisation*, where BPE leaves $\sim$717 Hindi slots unused against Unigram's 417. **Why 16K:** at d=512 with tied embeddings it costs 8.19M parameters against 32K's 16.38M, while the 32K models compress only $\sim$4.5% better. The 256 byte-fallback tokens are why the unknown-token rate is exactly zero.

3 Phase 2 --- Model and Pretraining

Both models are decoder-only Transformers implemented from primitive PyTorch layers only (`nn.Linear`, `nn.Embedding`, `nn.LayerNorm`, `nn.Dropout`) — no `nn.Transformer` classes and no pre-built attention. **Causal masking was verified, not assumed:** for every position t the test perturbs all later tokens and asserts the logits at $0 \ldots t$ are *bitwise* identical, then asserts position $t{+}1$ did change so it cannot pass vacuously. Attention mass above the diagonal is exactly 0.0, and disabling the mask makes the test fail immediately.

Setting	Model H	Model L
d_{model} / layers / heads	512 / 6 / 8	
Feed-forward width	1536 ($3\times$)	
Dropout / tied embeddings	0.1 / yes	
Context length	**512**	**1024**
Token embedding	8,192,000	8,192,000
Position embedding	262,144	524,288
Transformer blocks	15,765,504	15,765,504
Total parameters	**24,220,672**	**24,482,816**

Table 3: Configuration. The two differ by exactly 262,144 parameters — precisely the extra position rows Model L needs. The Transformer stacks are identical to the parameter.

Of three configurations within 0.5% of 24.2M parameters we chose d=512, 6 layers, d_{ff}=1536, because depth buys more than feed-forward width at this scale. **Weight tying** reuses the token embedding as the output projection, saving 8.19M parameters — the difference between a 6-layer stack and about a 3-layer one.

The one asymmetry is context length. Hindi documents have a median of 64 tokens and 99.3% fit in 512; Sindhi documents have a median of 219 and only 74.5% fit, since Sindhi text is long-form books where a 512-token window often sits *inside* one document. Fertility is nearly identical, so this is not a tokenizer artefact. Because longer context lowers loss for free, **Model L is evaluated at both 1024 and 512, and every cross-model claim here uses the matched 512 figures.**

Positional embeddings are learned absolute, not sinusoidal, RoPE or ALiBi: it is the standard GPT-2 baseline, the simplest thing to implement correctly from primitives, and it keeps positions as one isolated component — which is what let the bonus ablation (§5) be a one-flag change. Two costs appeared later: no extrapolation past the trained context length, and left-padding becomes unsafe at evaluation, since shifting a prompt rightward gives every token an untrained position, so the reasoning evaluator uses exact-length buckets and pads nothing.

Optimisation and hardware. Identical for both: AdamW $(0.9, 0.95)$, peak learning rate $6e{-4}$ cosine-decayed to $6e{-5}$ after 500 warmup steps, weight decay 0.1 on matrices only, gradient clipping 1.0, effective batch **24,576 tokens/step for 20,346 steps** (seed 1337). Both consumed **500,023,296 tokens** — 99.1% of the Hindi training corpus but only 80.9% of the Sindhi, 500M being the largest budget both supply without repetition and near compute-optimal for 24M parameters. Model H trained on a Kaggle T4 (fp16) for **11.94 h**, Model L on an RTX 3050 (bf16) for **7.66 h**. Resume was verified by hard-killing a run mid-training and confirming bitwise-identical weights.

Did the loss converge? Across 41 validation evaluations, validation loss **never once increased** for either model (0 rises out of 40 transitions), the best checkpoint is the final one in both cases, and the cosine schedule ran to completion at its floor. It had not flattened, though: over the last five evaluations loss was still falling at 0.028 nats per 100M tokens (H) and 0.038 (L). **Training stopped because the token budget ran out, not because learning stopped** — the correct outcome at a compute-optimal ratio.

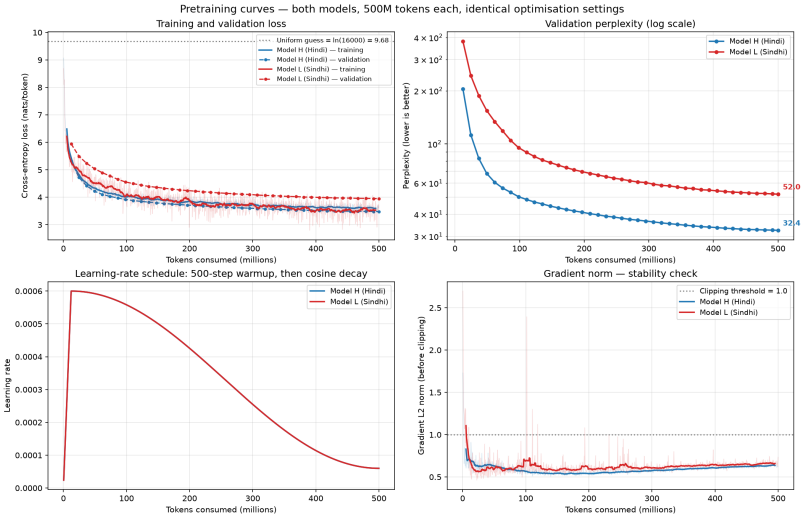


Figure 1: Pretraining curves for both models: loss, perplexity, learning rate and gradient norm against tokens consumed.

3.1 Perplexity and bits per byte

Perplexity is per *token*, and our two models have independently learned tokenizers, so one token means a different quantity of text in each. **Bits per byte normalises by underlying UTF-8 bytes and is tokenizer-independent**, which makes it the only honest way to compare them. On Sindhi, perplexity claims test is $2.6\times$ easier than validation; BPB says the opposite and correct thing. Investigating that revealed **encoding-corrupted text (mojibake)** in both test splits — legacy non-Unicode fonts decoded as Unicode, giving valid script characters in meaningless order. It affects 7.3% of the Hindi test split and **16.6% of the Sindhi**, against 0.6% and 1.6% of validation; being short and repetitive it is easy to predict, which deflated per-token perplexity wherever it was dense. The Phase 1 script filter could not catch it, because mojibake *is* valid Devanagari or Arabic-block text in nonsense order. **The**

Model	Split (ctx)	Loss	PPL	BPB
H	val (512)	3.4929	32.88	**0.5056**
H	test (512)	3.5722	35.59	**0.5375**
L	val (1024)	3.9541	52.15	**0.8821**
L	val (512)	3.9960	54.38	**0.8915**
L	test (1024)	2.9940	19.97	**0.9503**
L	test (512)	3.0194	20.48	**0.9584**

Table 4: Every token is scored exactly once using non-overlapping windows, so these are exact values rather than sampled estimates.

headline survives it: the $\sim 10\times$ cleaner validation splits give $+76.3\%$ against $+78.3\%$ on test. Model H picks the exact next token **35.85%** of the time and Model L **28.37%**, against 0.006% chance.

3.2 Generation

200 prompts of 64 tokens per model were continued for 128 tokens and scored against the true continuation. **Greedy** always takes the most likely token; **temperature** rescales the logits before sampling, <1 sharpening towards likely tokens and >1 flattening towards randomness.

	Setting	BLEU	chrF++	ROUGE-L	Dist-2	Rep.
H	greedy	1.46	10.88	0.089	0.122	0.810
	T=0.5	**1.78**	15.92	**0.115**	0.381	0.352
	T=1.0	1.24	**19.72**	0.107	0.824	0.008
	T=1.5	0.19	16.63	0.054	0.988	0.000
L	greedy	1.60	10.93	0.085	0.106	0.826
	T=0.5	**1.87**	13.92	**0.100**	0.288	0.513
	T=1.0	1.42	**19.50**	0.099	0.858	0.012
	T=1.5	0.34	16.49	0.045	0.994	0.000

Table 5: Generation quality. *Dist-2* is the fraction of distinct bigrams; *Rep.* the fraction of 4-grams repeating within one sample.

BLEU counts exact word-sequence overlap; **chrF++** works on character n-grams and tolerates the morphological variation that dominates both languages, so an inflected form differing by one suffix is a total miss for BLEU but largely correct for chrF — which makes chrF++ the more trustworthy of the two here. Both are low in absolute terms, expected when open-ended continuation has no single correct answer. **Greedy decoding degenerates**: over 80% of 4-grams repeat within one sample, and on a medical-text prompt Model H emits □□□□□□□ 12 □□ □□□□□□ □□ का प्रभाव गुर्दे पर क्या होता है? four times over, whereas T=1.0 gives varied, locally fluent but factually unmoored prose. T=1.5 **is too hot**: a repetition rate of exactly 0.000 alongside 0.99 distinct bigrams is the signature of noise, not creativity.

3.3 Attention and the generalisation gap

Attention was measured on 64 held-out windows of 256 tokens per model, excluding query positions below 32, since under a causal mask an early query is *forced* to be short-range. **Both models independently learned the same three-phase structure with depth**: layers 0–1 survey broadly, layers 2–4 specialise into sharp short-range heads, and layer 5 broadens again. Both converged on a near-pure **previous-token head** (Model H's L3H7 at 94% of its mass on position $t-1$, Model L's L4H4 at 96%) — two separately trained models finding the same component is evidence it is a genuinely useful solution. **Both show an attention sink**: 16.9% (H) and 23.8% (L) of all mass lands on position 0 regardless of what token sits there, because softmax forces every row to sum to 1.

The generalisation gap is the key Phase 2 finding. Model H ends with train loss 3.6018 against validation 3.4795, a gap of $-\mathbf{0.12}$ — none at all, since validation sitting below training loss is normal when training loss is measured on noisy single batches with dropout active. Model L ends at 3.5286 against 3.9506, a gap of $+\mathbf{0.42}$: it fits its training data slightly *better* than Model H fits its own, yet generalises considerably worse. That is the signature of a narrow, repetitive corpus and **the strongest single piece of evidence that the resource difference**

is about quality and diversity, not quantity. Two signals agree: Model L parks $1.4\times$ more mass on the sink and repeats far more under sharpened decoding (0.513 vs 0.352 at $T{=}0.5$).

4 Phase 3 --- Reasoning Finetuning

Following the brief, the focus is **comparative and basic reasoning, especially transitive and ordering comparisons**, with all text in the target language's own script and phrasing rather than English prompts wearing foreign names.

Family	Example (Hindi), answer in bold
Numeric comparison (2–3 entities)	राम की उम्र 25 साल है। श्याम की उम्र 30 साल है। किसकी उम्र ज्यादा है? → **श्याम**
Tie	two equal values → **बराबर** ("equal")
Chain, largest / smallest	राम की उम्र श्याम से ज्यादा है। श्याम की उम्र मोहन से ज्यादा है। सबसे ज्यादा उम्र किसकी है? → **राम**
Chain, endpoint relation	...राम की उम्र मोहन से ज्यादा है या कम? → **ज्यादा**

Table 6: The four question types, over five attributes (age, height, weight, price, population) across people, cities and objects. Only the last requires composing relations across the chain.

Numbers. 50,000 examples per language, from **17 surface templates per language, 8 written by the author** and converted by a helper script; 5 of the 17 are held out. 50,000 allows three stable epochs without inviting memorisation, and ties are held at $\sim$10% of every split, so "equal" is a real answer but never an easy default.

Split (n)	Held out	What it checks
`train` (40,000)	—	—
`val` (2,000)	—	model selection only
`test_iid` (2,000)	nothing	in-distribution accuracy
`test_entity` (2,000)	entity names	whether names were memorised
`test_template` (2,000)	5 templates	whether phrasing was memorised
`test_hop` (2,000)	problem size	extending 2–3 entities to 4–5

Table 7: Each test split removes exactly one thing, so a failure points at a specific cause.

Natural phrasing, established by measurement. The author is not a fluent Sindhi speaker, so **every word and construction was counted in that language's own pretraining corpus before being admitted**. This changed the design: the intended backbone was the adjective comparative ("X is *taller* than Y"), which the corpus rejected as nearly absent from both languages (کان ڳرو "heavier than" occurs 3 times in the Sindhi validation split). The dataset uses the dominant quantifier form instead (से ज्यादा 3,676; کان وڌيڪ 6,930), which also removes gender agreement from the backbone templates, since those quantifiers are invariant where the adjectives inflect. **Limitation:** the words are attested, but their composition into sentences is the author's own and was not reviewed by a native speaker.

Leakage prevention and verification. Leakage is prevented *by construction*: held-out names are fixed in committed lexicon files before generation, every text is deduplicated globally across all six splits, and the generator asserts eight conditions at build time, all returning zero — no verbatim train/test overlap, no held-out name in train, no held-out template or problem size shared with train. Because values are drawn fresh per example, a structural "twin" in training carries no usable answer: about 60% of test items have one, but **copying its answer scores only 32.6%**, at or below chance, so the 78.8% cannot come from recall. The test splits were never used for any decision, and balance controls were measured — the first build put 58.6% of one split's answers in first position, fixed by stating each chain link from a randomly chosen end.

4.1 Finetuning approach

Each model is finetuned from **its own** pretrained checkpoint using **its own** tokenizer, unchanged. **The loss is computed on the answer tokens only** (the prompt is masked with `ignore_index=-100`), so the model is graded

on answering rather than on reciting the question back. Padding is right-aligned, safe under a causal mask because a supervised position can never attend forward to a pad; a unit test asserts this.

Setting	Both models
Learning rate	1e−4 peak, cosine to 1e−5
Warmup / optimizer	100 steps / AdamW (0.9, 0.95)
Weight decay / clip	0.01 / 1.0
Batch / length	32 × 96 tok / 3 epochs (3,750 steps)
Loss / seed	answer tokens only / 1337
LR probe (1 epoch, validation only), mean val. accuracy	
5e−5 / **1e−4** / 3e−4	39.3% / **44.0%** / 39.0%
Best checkpoints	H step 3600, L step 3750

Table 8: Finetuning settings, identical for both models, so nothing in the comparison is a hyperparameter artefact.

The learning rate was chosen on validation data only, for both models at once. Three rates ran for one epoch each on both languages, and 1e−4 won for *each* language independently, so sharing one rate cost neither model anything — whereas choosing on Hindi alone would have tilted the comparison towards the higher-resource model. The metric throughout is **exact-match accuracy**. **Learning has a delayed onset**: both models sit at the chance floor for roughly the first epoch, then take off, Model H around step 1,200 and Model L about 400 steps later. Model L's best checkpoint is its *last*, so the H–L gap below is a comparison at an equal budget.

4.2 Results

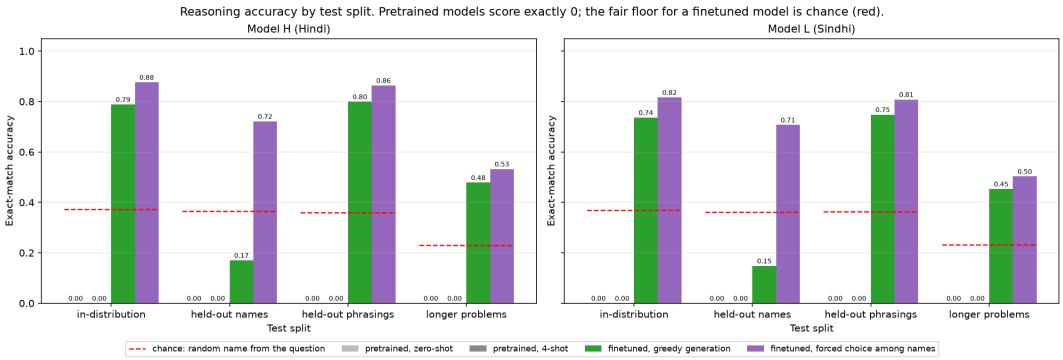


Figure 2: Reasoning accuracy against the chance floor — what a model scores by naming a random entity from the question.

Split	Model H	Model L	Chance
Pretrained (0- and 4-shot)	0.0%	0.0%	—
In-distribution	**78.8%**	**73.7%**	37.2 / 36.8%
Held-out phrasings	80.0%	74.8%	35.9 / 36.4%
4–5 entities (trained 2–3)	47.9%	45.3%	23.0 / 23.2%
Held-out names	17.0%	14.8%	36.5 / 36.2%
Held-out names, forced choice among names in the question			
Finetuned	**72.2%**	**70.7%**	40.7 / 40.6%
Pretrained	38.8%	39.7%	40.7 / 40.6%

Table 9: Test accuracy by split under greedy decoding, plus forced choice on the held-out-name split.

The baseline is chance, not zero. The pretrained models score exactly 0.0% in all 16 conditions because they never produce an answer at all: given a question about city populations, Model H continues the *premise* pattern and writes another fact — कोलकाता की आबादी 34 लाख है। Four worked examples change the *shape* of the output but

not its content, since at 24M parameters these models cannot infer a task from examples. Reporting zero as the baseline would overstate what finetuning taught: a model knowing only the answer *format* scores about 36% by naming a random entity, and every run sat there for its first ∼1,000 steps. **The honest gain is about 42 points above chance for Model H and 37 for Model L.**

New phrasings: fully — held-out templates score as well as trained ones. **Longer problems: partly** — roughly twice chance on 4–5-entity problems never seen in training. **Closer numbers are harder**, the key positive result: accuracy runs from 98.7% (H) and 97.5% (L) when values differ by more than 40% of their range down to 51.9% and 46.3% when they differ by under 2%. A surface heuristic would not care how close the numbers are, so this is the strongest evidence that the models genuinely compare magnitudes (`figures/reasoning_what_is_learned.png`).

Task type	Model H	Model L	Chance
Numeric comparison, no tie	**87.1%**	**84.0%**	29.1%
Numeric comparison, tie	43.1%	36.4%	30.2%
Chain, which is largest	**68.0%**	41.9%	33.3%
Chain, which is smallest	**72.3%**	45.6%	33.3%
Chain, endpoint relation	40.9%	50.5%	50.0%

Table 10: Accuracy by task type, in-distribution split only.

The H–L gap sits almost entirely in multi-step problems: within 3 points on comparing two numbers, about 26 points apart on chains, where Model L barely clears chance. **Neither model learned the endpoint relation question** — both predict "more" and "less" in balanced proportions but independently of the correct answer, Model H actually *below* the 50% chance line. That qualifies the chain result: "which is largest" can be solved without composing relations, since the largest entity is simply the one never named on the smaller side of any link. The endpoint question is the one genuinely requiring an entity to be bound to its place in the order, and both fail it, so **Model H's chain accuracy is not by itself evidence of transitive composition.**

Held-out names: not the failure it looks like. On unseen names accuracy collapses to 17.0% and 14.8%, *below* chance. The outputs explain it: for Model H, 67.4% are बराबर ("equal") or another relation word, 21.5% are a *training* name absent from the question, and only 1.0% are a wrong name that does appear. The model is not choosing badly among candidates; it is not producing them at all, and tokenization was ruled out since both name sets average about one token. Under **forced choice**, where every candidate is scored by log-probability instead of generated, accuracy on those same questions is **72.2%** and **70.7%**, against 87.6% and 81.7% on seen names. **The comparison does transfer to unseen names**, degrading by only 11–15 points, while about 55 points are lost purely to *producing* the answer. "Equal" dominates the failures because it is the most frequent training answer at ∼11%, covering every tie while names are spread over ∼50 entities.

4.3 Attention after finetuning

The Phase 2 toolkit was re-run on both checkpoints over identical inputs (Figure 3). **A selector head appears**: at the answer position, the strongest head's attention to the correct entity against the average distractor goes from 0.185 vs 0.173 — no preference at all — to **0.687 vs 0.119** for Model H and 0.667 vs 0.157 for Model L. **It works just as well on unseen names** (0.700 vs 0.110), independently confirming the finding above: the model *looks at* the correct unseen entity and still cannot emit it, so the failure lies downstream of attention, in the output layer. With depth, a layer-1 head sends ∼80% of the answer position's attention to the *digit* tokens, and by layer 5 it has moved to the *entity names* ($0.267 \rightarrow 0.785$).

On ordinary text, local attention was dismantled. Entropy rose in every layer and middle-layer distances grew two- to six-fold. The previous-token heads were *destroyed rather than repurposed*: Model H's L3H7 fell from 0.937 to 0.334 previous-token mass and Model L's L4H4 from 0.965 to 0.217, with the count of heads holding more than half their mass on the previous token going from 2 to **0** in both.

4.4 What finetuning cost, and whether it was worth it

This is a genuine trade-off, not a free win. On the same held-out corpus text as Phase 2, bits per byte went from 0.5375 to **1.0111** for Model H ($+88.1\%$) and from 0.9584 to **1.9563** for Model L ($+104.1\%$); both roughly doubled, and the gap between them *widened* from $+78.3\%$ to $+93.5\%$. The finetuned checkpoints are task-specialised models, not improved ones: what was bought is 78.8% and 73.7% on a task where both previously scored zero, and what was sold is roughly half the general language-modelling quality, with the lower-resource model paying

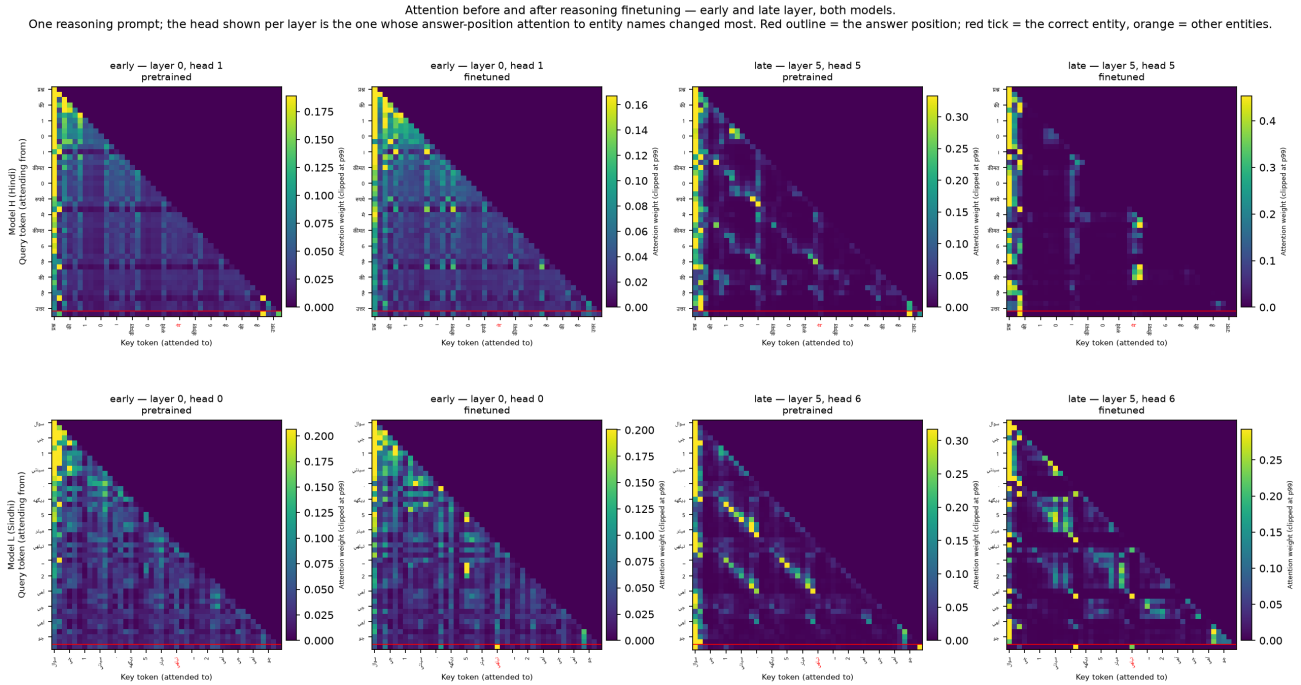


Figure 3: Pretrained against finetuned attention on one reasoning prompt, for an early (0) and a late (5) layer of both models. Each panel shows the head in that layer whose answer-position attention to entity names changed most. Red outline: the answer position; red tick: the correct entity. All four heads per layer: `figures/attention_finetune_{hindi,sindhi}_layer{0,5}.png`.

more. **For this project that is worth it**, since the question was whether models of this size can learn structured comparative reasoning at all; **for a deployed system it would not be**, because a model that answers comparison questions but has lost half its fluency is worse than either a general model or a purpose-built one.

An ablation that refuted our own explanation. We first blamed the loss mask: with loss on the answer only and embeddings tied, every step reshapes all 16,000 token embeddings towards a tiny answer vocabulary. Finetuning again with loss on *every* token, all else identical, **did not help** — Model L was unchanged (validation BPB $1.8005 \rightarrow 1.8066$) and Model H became far *worse* ($0.9739 \rightarrow 2.3620$) while also losing task accuracy. The two losses break the model in *different* ways: the first predicts end-of-document everywhere, the second turns it into a confident generator of reasoning questions. The real cause is finetuning on a narrow templated distribution with nothing preserving the general one; the loss mask only decides what the model collapses *into*. Replay is the obvious untested remedy.

Model H, correct (`test_template`, held-out phrasing)
प्रश्न: मुंबई की आबादी 156 लाख है। पटना की आबादी 161 लाख है। कोलकाता की आबादी 216 लाख है। कौन ज्यादा आबादी का है? उत्तर: **कोलकाता** ✓
Model L, correct (`test_iid`)
✓ **شاهد** سوال: نديم جي ڀيڳهه 105 سينٽي ميٽر آهي. شاهد جي ڀيڳهه 142 سينٽي ميٽر آهي. نديم ۽ شاهد مان ڪنهن جي ڀيڳهه وڌيڪ آهي؟ جواب:
Model H, wrong (`test_entity`, unseen name): expected माया, produced **बराबर** (“equal”) — the default-answer failure above

Table 11: Representative outputs, sampled with a fixed seed rather than hand-picked. More in `report/phase3_results.md` §7.

5 Bonus --- Removing Positional Embeddings

The Hindi model was retrained from scratch with **no positional embeddings at all**. In theory this should hurt badly: self-attention is permutation-invariant, so without positional information a Transformer sees a *bag* of tokens and राम श्याम से बड़ा है would be indistinguishable from श्याम राम से बड़ा है. The causal mask leaks only a weak, indirect cue, since position t can attend to exactly $t{+}1$ tokens — so **we expected a large degradation**. **Implementation:** a single config flag, `use_positional_embeddings`; when false the position table is never created.

Corpus, tokenizer, schedule, batch, seed and token budget are identical to Model H, and the ablated model has 23,958,528 parameters, exactly 262,144 fewer.

	Standard	No positions	Change
Parameters	24,220,672	23,958,528	−262,144
Best validation loss	3.4795	3.5378	+0.058
Perplexity (test)	35.59	37.83	+6.3%
BPB (test, 512)	**0.5375**	**0.5467**	**+1.7%**
BLEU (greedy)	1.46	1.08	−25.8%
chrF++ (greedy)	10.88	10.02	−7.9%
chrF++ (T=1.0)	19.72	19.94	+1.1%

Table 12: Bonus ablation. Full tables: `report/nopos_ablation.md`.

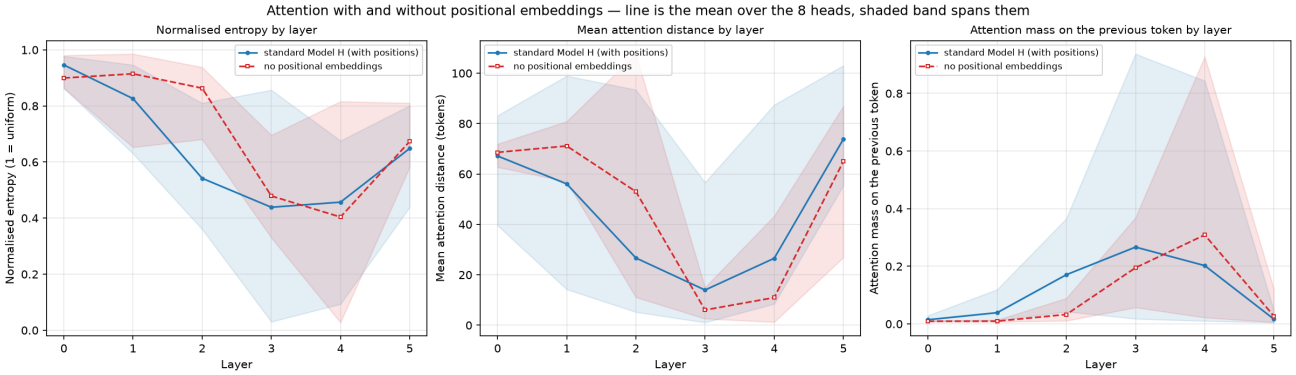


Figure 4: Attention with and without positional embeddings, by layer: normalised entropy, mean attention distance and mass on the previous token. Line = mean over the 8 heads, band = their range. Heatmaps: `figures/attention_nopos_layer{0,5}.png`.

What we found. The penalty is real, consistent — and **much smaller than theory predicts**: removing positional embeddings entirely costs only **1.7% in bits per byte**. The gap opens early and stays almost flat (+0.073 at a quarter of training, +0.058 at the end). Figure 4 shows *how* the ablated model copes: it rebuilds a positional mechanism from the only cue it has. **It still finds a near-pure previous-token head** (L4H3 at 0.927, against the standard model's L3H7 at 0.937) and still ends with exactly 2 such heads; what changes is where the work happens, with early layers much more diffuse (layer 2 entropy $0.542 \rightarrow 0.863$) and middle layers *sharper and more local* than the standard model's (layer 3 distance $14.0 \rightarrow 6.1$).

Conclusion. Learned absolute positional embeddings are worth their 262,144 parameters, but at this scale they are a convenience rather than a necessity: a causal decoder recovers most of what it needs about word order from the mask alone, because the mask makes each position's visible context a different size. The effect is clearest under greedy decoding (BLEU −25.8%) and vanishes at T=1.0. **One caveat we cannot remove:** Model H trained on a Kaggle T4 in fp16 while the ablation ran on the RTX 3050 in bf16, so hardware and precision differ alongside the ablation.

6 Final comparison --- the brief's four questions

1. How did data scale and quality differ? *Scale did not favour Model H:* Sindhi had *more* training text (618.4M tokens against 504.8M), and both trained on the same 500M, so Model L used 81% of its corpus and Model H 99%. *Quality and diversity did.* Hindi came from broad web-scale corpora spanning many domains, Sindhi from a handful of book and newspaper repositories — more tokens from a narrower distribution. Sindhi also carries far more encoding-corrupted legacy-font text (16.6% of its test split against 7.3%), a direct consequence of less clean digital text existing for the language, and its documents are 3.4× longer at the median.

2. How do language-modelling and reasoning results compare across the tiers? The gap is **large for language modelling and uneven for reasoning**. Model L needs +78.3% bits per byte, rising to +93.5% after finetuning. On reasoning it is only 5.1 points behind in-distribution, 3.1 behind on plain numeric comparison and 1.5 behind

under forced choice on unseen names — but about 26 points behind on chains, where it barely clears chance. On simple comparison the lower-resource model is nearly as capable; on multi-step problems it falls well behind and learns later.

3. What tokenizer / corpus factors most affected the lower-resource model? *Not the tokenizer:* both use 16K Unigram vocabularies with near-identical fertility, zero unknown tokens and entity names averaging about one token, so tokenization explains neither the language-modelling gap nor the reasoning results — and the one tokenizer signal points *away* from scarcity, since Sindhi's vocabulary utilisation is *higher* (98.2% vs 97.4%), consistent with a narrower distribution of text. *The corpus*, in order of evidence: narrow source diversity (best supported, via the generalisation gap); encoding corruption, which passed the Phase 1 filters because corrupted legacy-font text is still valid Arabic-block characters; and long documents, which forced the 1024-token context and whose effect was controlled by evaluating both models at 512.

4. What evidence explains the observed differences? The **generalisation gap** is the strongest: Model L fits its own training data *better* than Model H fits its own (3.53 vs 3.60) yet generalises worse ($+0.42$ against -0.12), the signature of a narrow, repetitive corpus. **Robustness to contamination** rules out mojibake as the cause, since the gap is $+76.3\%$ on the far cleaner validation split against $+78.3\%$ on test. **Slower, less complete reasoning** — Model L's onset is later and its chain accuracy near chance while its simple comparisons nearly match — is consistent with a weaker language model limiting composition rather than basic comparison. **Greater fragility**: Model L lost more language modelling to finetuning ($+104\%$ vs $+88\%$) and its attention was disrupted more, though that link is correlational. **What it is not:** capacity (identical Transformer stacks), budget or optimisation (identical), or the tokenizer.

7 Limitations, deliverables and closing

Limitations. Composed sentences were not checked by a native Sindhi speaker; both models were still improving when training stopped, so the H–L reasoning gap is at an equal budget and may overstate the eventual difference; the learning-rate probe ran only one epoch; the finetuned checkpoints are task-specialised, not improved, and replay was not tested; attention findings are correlational; there was no hyperparameter search beyond the learning rate and one run per condition; and the contaminated test splits were measured but not cleaned, with cross-split fuzzy deduplication not run.

Deliverables. Code for data, tokenizers, the Transformer, training, finetuning and evaluation is in the repository, with per-phase reports, all figures and tables under `report/`. Google Drive links for the corpora, tokenized data and the pretrained, finetuned and ablated checkpoints are in the top-level `README.md`, with step-by-step reproduction commands for all three phases and the bonus.

Closing. The most useful findings came from distrusting our own numbers: a perplexity that looked too good exposed 16.6% corrupted text and moved the headline metric to bits per byte, an accuracy that looked catastrophic turned out to be a generation failure on top of an intact ability, and an explanation we found convincing was refuted by our own ablation. Both models went from literally zero to roughly 40 points above chance, generalising fully across new phrasings, partly across longer problems, and — when asked to choose rather than to write — across entirely new names. But neither learned the one question requiring composition across a chain, so the honest claim is *comparison and elimination*, not transitive reasoning. Next steps: replay of ordinary corpus text during finetuning, a longer training budget, and cleaning the mojibake from the test splits before recomputing.